

# Ускорение обучения. Подготовка к деплою

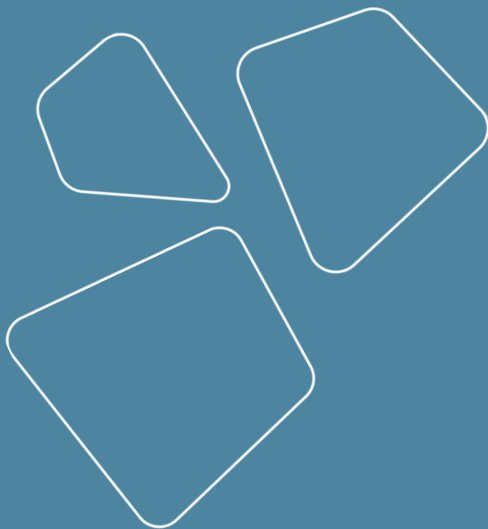
Vladislav Goncharenko

ITMO, fall 2024



# Outline

- Mixed Precision Training и тензорные ядра
- TensorRT



# Mixed Precision

Ускоряем обучение

---

girafe  
ai

01

# TLDR

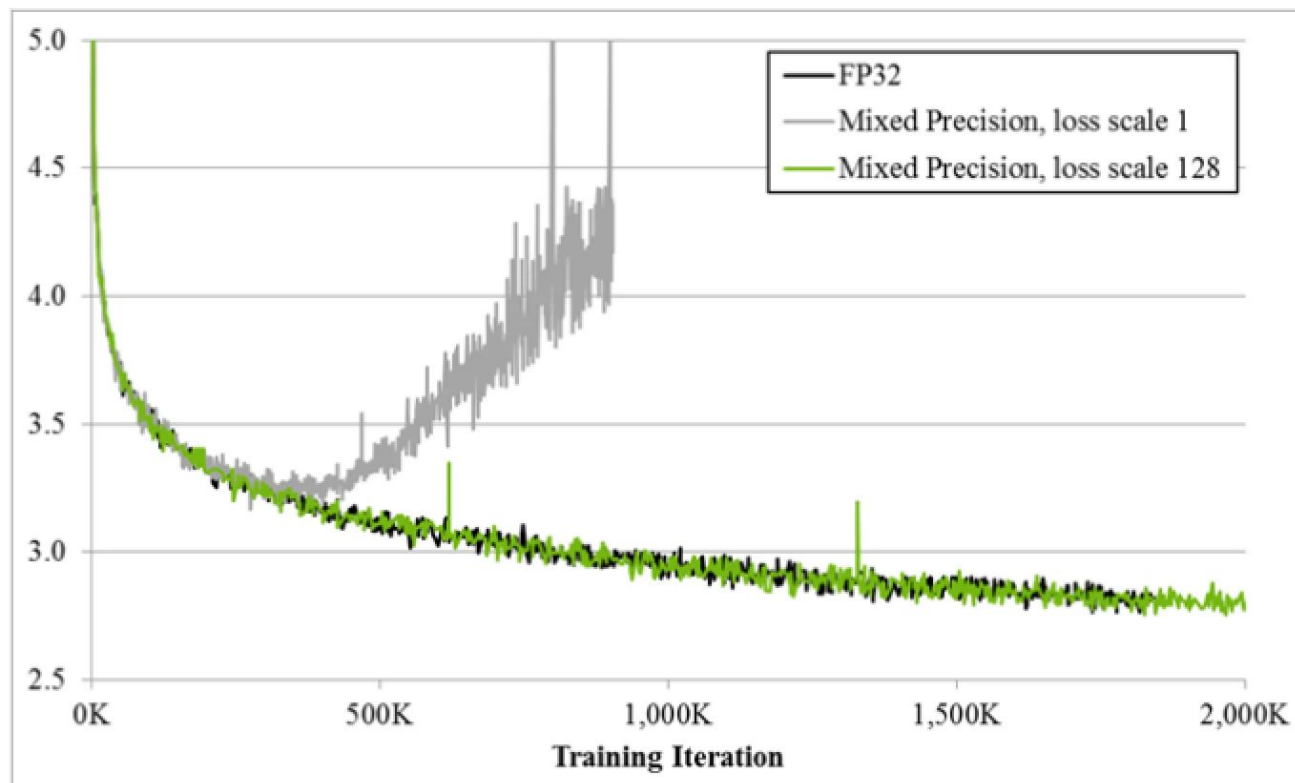
## **Делаем:**

- Заставляем модель использовать FP16 там, где это выгодно
- Добавляем Loss Scale для борьбы с затуханием градиента

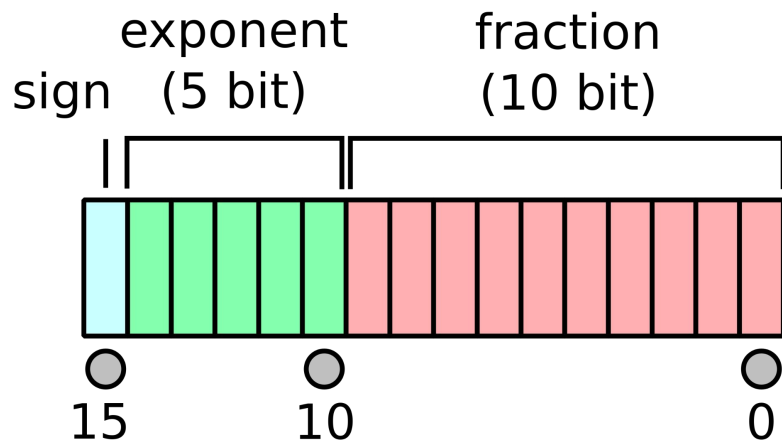
## **Получаем:**

- Более быстрое обучение, ускорение до 8x раз (нет)
- Уменьшение количества необходимой видеопамяти
  - То есть можем использовать батчи большего размера
  - Меньше оверхед на память

# Пример

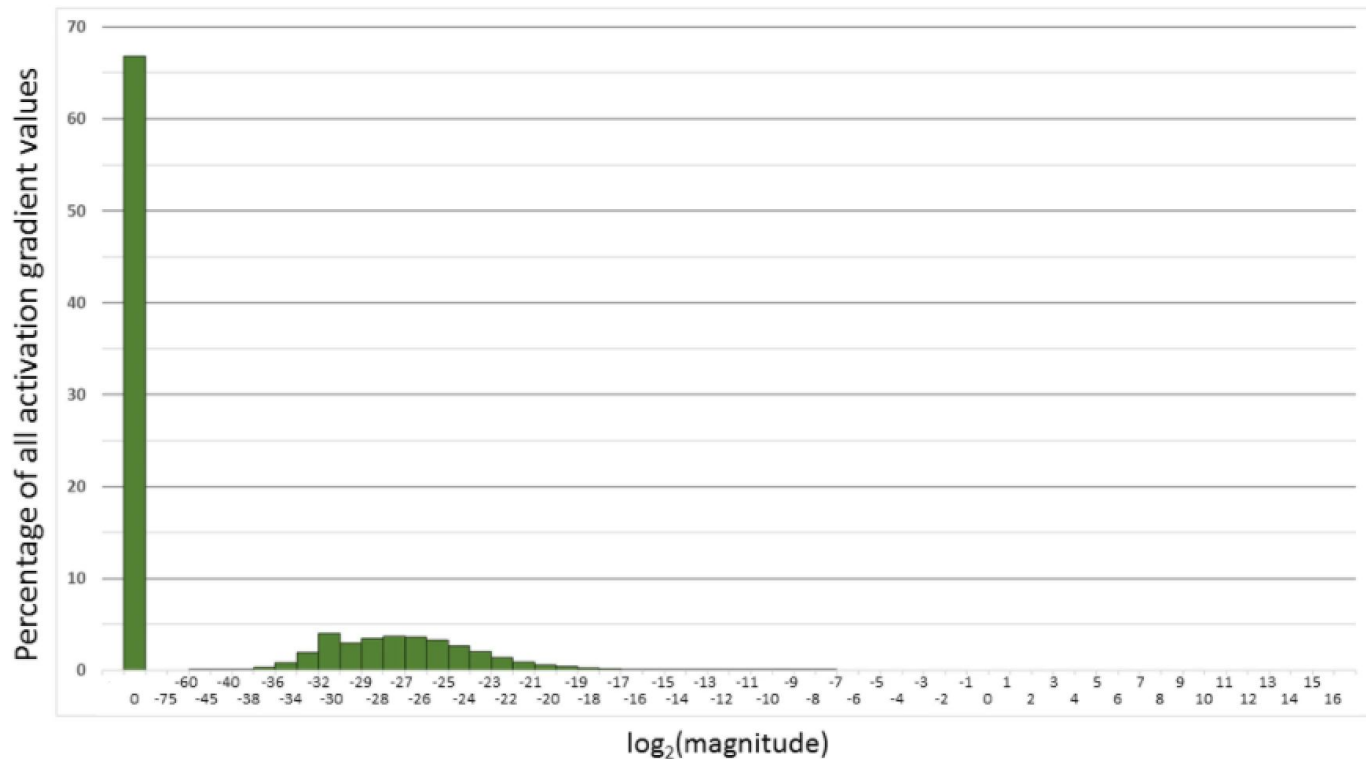


# Half Precision

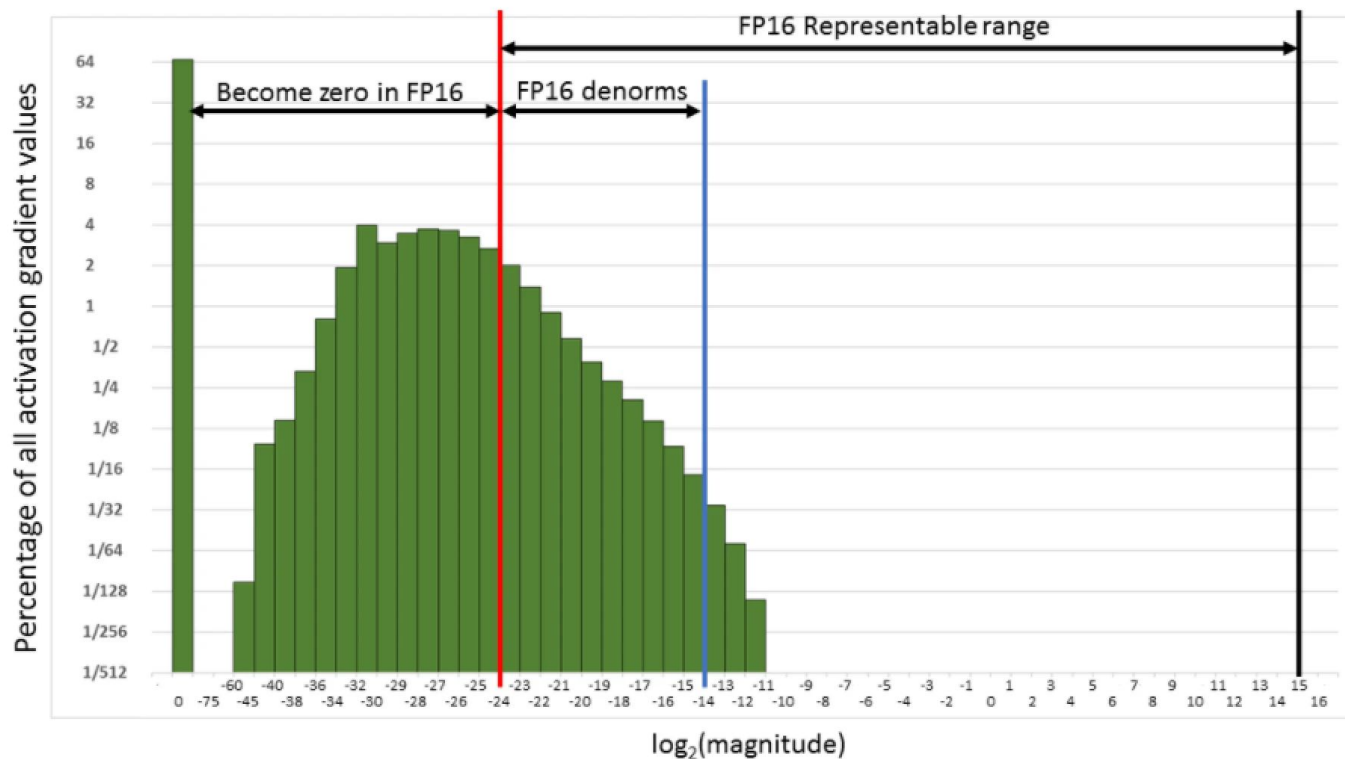


- Normalized:
  - от  $2^{(-14)}$  до  $2^{(15)}$
- Denormalized:
  - от  $2^{(-24)}$  до  $2^{(-15)}$
- Максимальное нормализованное:
  - **65504**
- Минимальное нормализованное:
  - **~0.00006**
- Минимальное **де**нормализованное:
  - **6e-8**

# Магнитуды градиентов FP32



# Магнитуды градиентов FP32





# Loss Scaling

1. Maintain a primary copy of weights in FP32.
2. For each iteration:
  - Make an FP16 copy of the weights.
  - Forward propagation (FP16 weights and activations).
  - Multiply the resulting loss with the scaling factor  $S$ .
  - Backward propagation (FP16 weights, activations, and their gradients).
  - Multiply the weight gradient with  $1/S$ .
  - Complete the weight update (including gradient clipping, etc.).

# Scaling Factor

## Статический подход:

- Выберем  $S$  так, что его произведение с максимальным абсолютным значением градиента было  $< 65504$

## Динамический подход:

- ???????

# Scaling Factor

1. Maintain a primary copy of weights in FP32.
2. Initialize  $S$  to a large value.
3. For each iteration:
  - a. Make an FP16 copy of the weights.
  - b. Forward propagation (FP16 weights and activations).
  - c. Multiply the resulting loss with the scaling factor  $S$ .
  - d. Backward propagation (FP16 weights, activations, and their gradients).
  - e. If there is an Inf or NaN in weight gradients:
    - i. Reduce  $S$ .
    - ii. Skip the weight update and move to the next iteration.
  - f. Multiply the weight gradient with  $1/S$ .
  - g. Complete the weight update (including gradient clipping, etc.).
  - h. If there hasn't been an Inf or NaN in the last  $N$  iterations, increase  $S$ .

# AMP - Automatic Mixed Precision

Steps:

- Converting the model to use the **float16** data type **where possible**.
- **Keeping float32 master weights** to accumulate per-iteration weight updates.
- Using **loss scaling** to preserve small gradient values.

# Тензорные ядра и AMP

1. **Satisfy** Tensor Core shape **constraints**
2. **Increase arithmetic intensity**
3. **Decrease** fraction of work in **non-Tensor Core operations**

# 1. Satisfy Constraints

## Matmul:

1. **FP16** - Gemm`s M, N, K кратны **8**
2. **INT8** - Gemm`s M, N, K кратны **16**

## Conv:

1. **FP16** - in\_channels, out\_channels кратны **8**
2. **INT8** - in\_channels, out\_channels кратны **16**

# 1. Satisfy Constraints

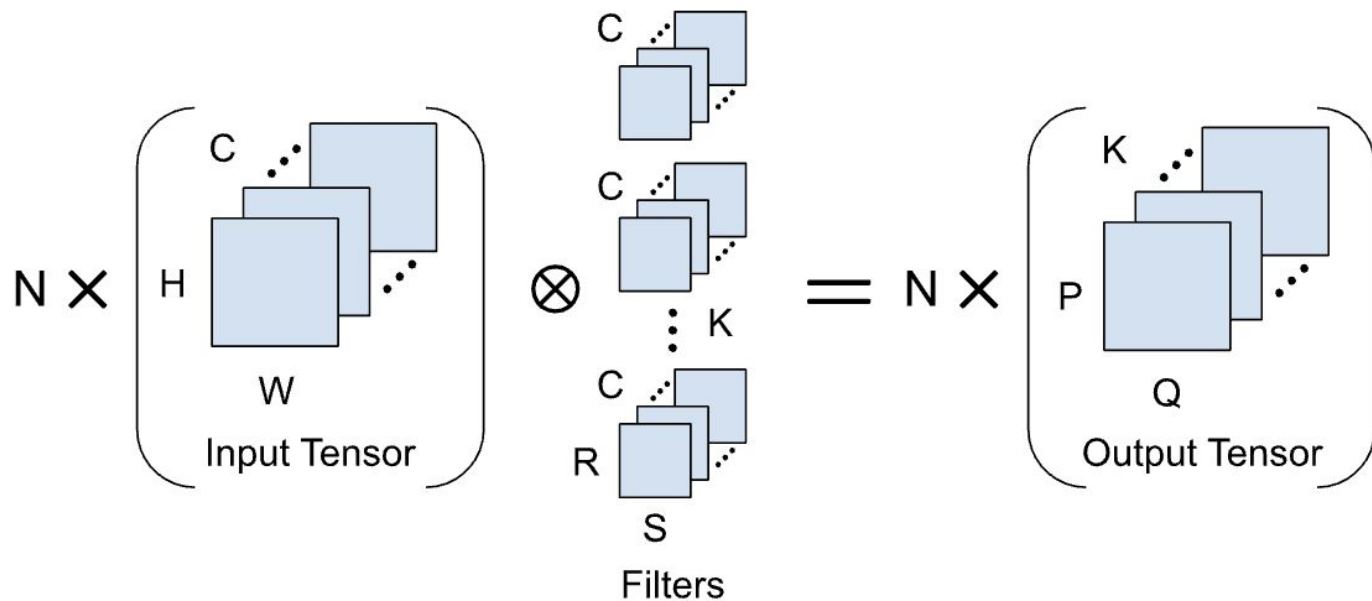
## Linear Gemm

Table 1. Mapping of inputs, outputs, and batch size to GEMM parameters M, N, K.

| Computation Phase   | M                 | N                 | K                 |
|---------------------|-------------------|-------------------|-------------------|
| Forward Propagation | Number of outputs | Batch size        | Number of inputs  |
| Activation Gradient | Number of inputs  | Batch size        | Number of outputs |
| Weight Gradient     | Number of inputs  | Number of outputs | Batch size        |

# 1. Satisfy Constraints

## Conv Gemm





## 2. Increase Arithmetic Intensity

Model architecture:

- Prefer dense math operations.
  - For example, vanilla convolutions have much higher arithmetic intensity than depth-wise separable convolutions.
- Prefer wider layers when possible.

# 3. Decrease Non-Tensor Work

Как считается ускорение:

$$1 / (x + ((1-x) / y))$$

- $x$  - доля времени в не Tensor-Core операциях
- $y$  - ускорение от использования тензорных ядер

Например, если *половина* времени обучения тратится на операции в тензорных ядрах, а в этих операциях ускорение равно 5, тогда общее ускорение от использования смешанной точности будет равно  $1 / (0.5 + (0.5 / 5)) = 1.67$

# ***Allow / Deny / Infer* Lists**

- **AllowList:**
  - Convolutions
  - Fully-connected layers
- **DenyList:**
  - Large reductions
  - Cross entropy loss
  - L1 Loss
  - Exponential
- **InferList:**
  - Element-wise operations (add, multiply by a constant)

TyT: [https://github.com/NVIDIA/apex/blob/master/apex/amp/lists/functional\\_overrides.py](https://github.com/NVIDIA/apex/blob/master/apex/amp/lists/functional_overrides.py)

# Hardware requirements

1. Наличие Тензорных Ядер :)

# Примеры ускорений

| Model Script                                       | Framework  | Data Set                   | FP32 Accuracy   | Mixed Precision Accuracy | FP32 Throughput                  | Mixed Precision Throughput       | Speed-up  |
|----------------------------------------------------|------------|----------------------------|-----------------|--------------------------|----------------------------------|----------------------------------|-----------|
| <a href="#"><u>BERT Q&amp;A</u></a>                | TensorFlow | SQuAD                      | 90.83<br>Top 1% | 90.99<br>Top 1%          | 66.65<br>sentences/sec           | 129.16<br>sentences/sec          | 1.94      |
| <a href="#"><u>SSD w/RN50</u></a>                  | TensorFlow | COCO 2017                  | 0.268<br>mAP    | 0.269<br>mAP             | 569<br>images/sec                | 752<br>images/sec                | 1.32      |
| <a href="#"><u>GNMT</u></a>                        | PyTorch    | WMT16<br>English to German | 24.16<br>BLEU   | 24.22<br>BLEU            | 314,831<br>tokens/sec            | 738,521<br>tokens/sec            | 2.35      |
| <a href="#"><u>Neural Collaborative Filter</u></a> | PyTorch    | MovieLens 20M              | 0.959<br>HR     | 0.960<br>HR              | 55,004,590<br>samples/sec        | 99,332,230<br>samples/sec        | 1.81      |
| <a href="#"><u>U-Net Industrial</u></a>            | TensorFlow | DAGM 2007                  | 0.965-0.988     | 0.960-0.988              | 445<br>images/sec                | 491<br>images/sec                | 1.10      |
| <a href="#"><u>ResNet-50 v1.5</u></a>              | MXNet      | ImageNet                   | 76.67<br>Top 1% | 76.49<br>Top 1%          | 2,957<br>images/sec              | 10,263<br>images/sec             | 3.47      |
| <a href="#"><u>Tacotron 2 / WaveGlow 1.0</u></a>   | PyTorch    | LJ Speech Dataset          | 0.3629/-6.1087  | 0.3645/-6.0258           | 10,843 tok/s<br>257,687<br>smp/s | 12,742 tok/s<br>500,375<br>smp/s | 1.18/1.94 |

# Performance Limitations

Пусть:

- $T_{mem}$  - время доступов к памяти
- $T_{math}$  - время выполнения арифметических операций

Суммарное время:  $\max(T_{mem}, T_{math})$ .

# Performance Limitations

Оценим:

- $T_{mem} = \#bytes / BW_{mem}$
- $T_{math} = \#ops / BW_{math}$

, тут  $\#bytes$  - количество необходимых байт в памяти,  $\#ops$  - количество мат операций,  $BW_{mem}$  - Memory Bandwidth, а  $BW_{math}$  - Math Bandwidth.

$$T_{math} > T_{mem} \Leftrightarrow \#ops / BW_{math} > \#bytes / BW_{mem} \Leftrightarrow$$

$$\Leftrightarrow \#ops / \#bytes > BW_{math} / BW_{mem}.$$

$\#ops / \#bytes$  -- это и есть арифметическая интенсивность

# Performance Limitations

Рассмотрим Nvidia V100:

- BW\_mem\_onchip = 3.1 Tb/s
- BW\_mem\_offchip = 900 GB/s
- BW\_math = 125 FP16 TFLOPS

=> **арифметическая интенсивность между 40 и 140** (в зависимости от источника данных для операции).



# Performance Limitations

Рассмотрим конкретный пример с полносвязанным слоем:

- $in = 4096, out = 1024, bs = 512$
- $FLOP = 2 * bs * in * out \approx 4 * 10^9 \text{ flop (4 GFLOP)}$ , (2 так как mul-асс операция)
- Bytes (weight) =  $in * out * 2$ , (2 байта тк fp16)
- Bytes (inputs) =  $bs * in * 2$
- Bytes (out) =  $bs * out * 2$
- Bytes (total) =  $in * out * 2 + bs * 2 * (in + out) \approx 0.01 * 10^9$

**#ops / #bytes = 400** => для nvidia V100 **ТОЧНО** ограничено арифметикой

# Performance Limitations

Рассмотрим пример с ReLU идущим за этим линейным слоем:

- $\text{out} = 1024, \text{bs} = 512$
- $\text{FLOP} = \text{bs} * \text{out} \approx 0.5 * 10^6 \text{ (MFLOP)}$
- $\text{bytes (input and output)} = \text{bs} * \text{out} * 2 + \text{bs} * \text{out} * 2 \approx 2 * 10^6 \text{ (MB)}$

=> **#ops / #bytes = 0.25** => на V100 операция ограничена памятью

# Performance Limitations

Попробуем взять тот же самый полносвязанный слой и оценить границу когда операция становится ограничена памятью: in = 4096, out = 1024

- $\text{\#ops} / \text{\#bytes} = 40$
- $40 = (2 * x * 1024 * 4096) / (1024 * 4096 * 2 + x * 2 * (1024 + 4096))$
- $x = \text{bs} \approx 42$

# ONNX format

---

girafe  
ai

02

# Definition, specification

TODO

# Runtimes overview

- ONNX runtime
- OpenVino
- CoreML
- TensorRT
- TorchScript
- etc.

# TensorRT

---

girafe  
ai

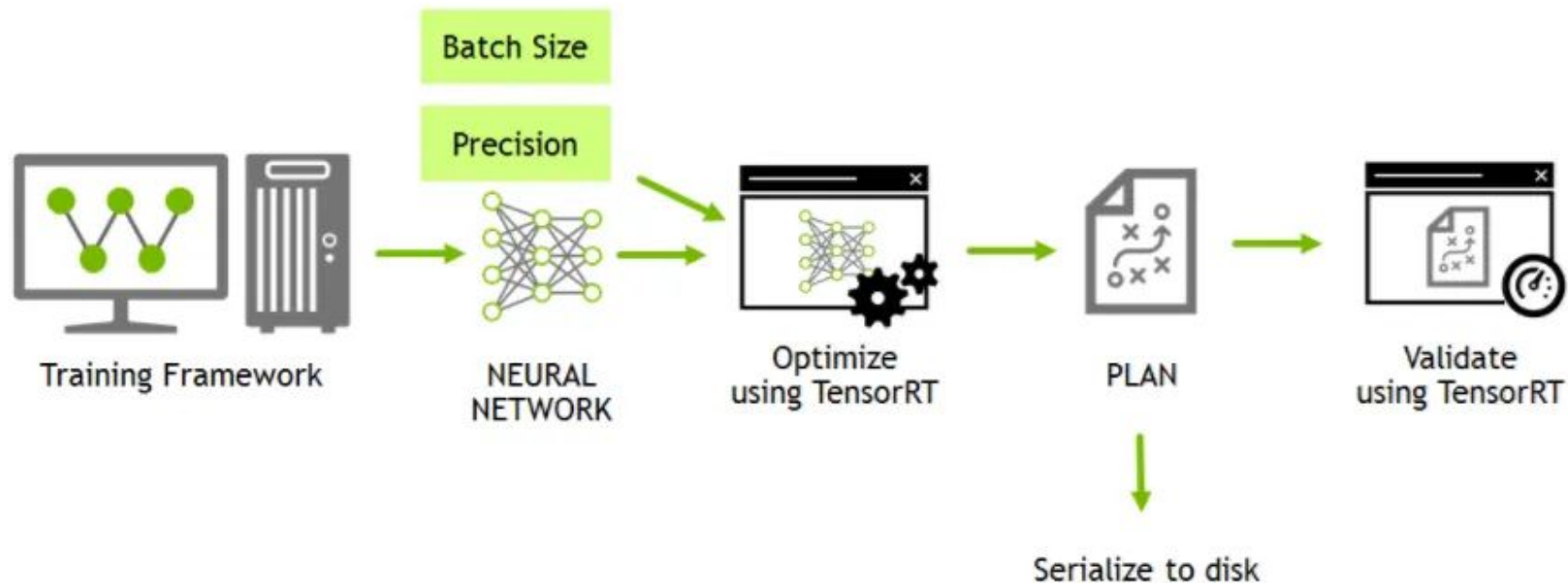
# 03

# Train vs Infer

- Железо
  - Много моделей деплоится на CPU
  - Выгодно учить на DGX с A100/V100 с SXM а не PCI-E
    - Но DGX-ы супер прожорливые, особенно в простое
      - 80W idle против 30W idle
    - Обычно не инферим с большим батчем, следовательно не нужно много прожорливой видеопамяти
- Тренировка - оффлайн, инференс - реалтайм
- Нужна высокопроизводительная инфра для обработки запросов
- Как минимум мы не считаем градиенты в инференсе
  - Хотя знаю одно исключение (grad-cam)
- Экономим электричество при инференсе
  - Электричество дорогое
  - В ДЦ вас могут рассчитывать по пику электропотребления
  - У видеокарт нулевой КПД => все электричество переходит в тепло, тепло сложно рассеивать
- Невероятно замороченная оптимизация для деплоя, так как даже 1% к метрике производительности на большом объеме данных может дать прирост в гри-сри-годы или даже столетия
- Большая дисперсия софтины для инференса. Надо заботиться об отказоустойчивости. Нужны балансировщики нагрузки...

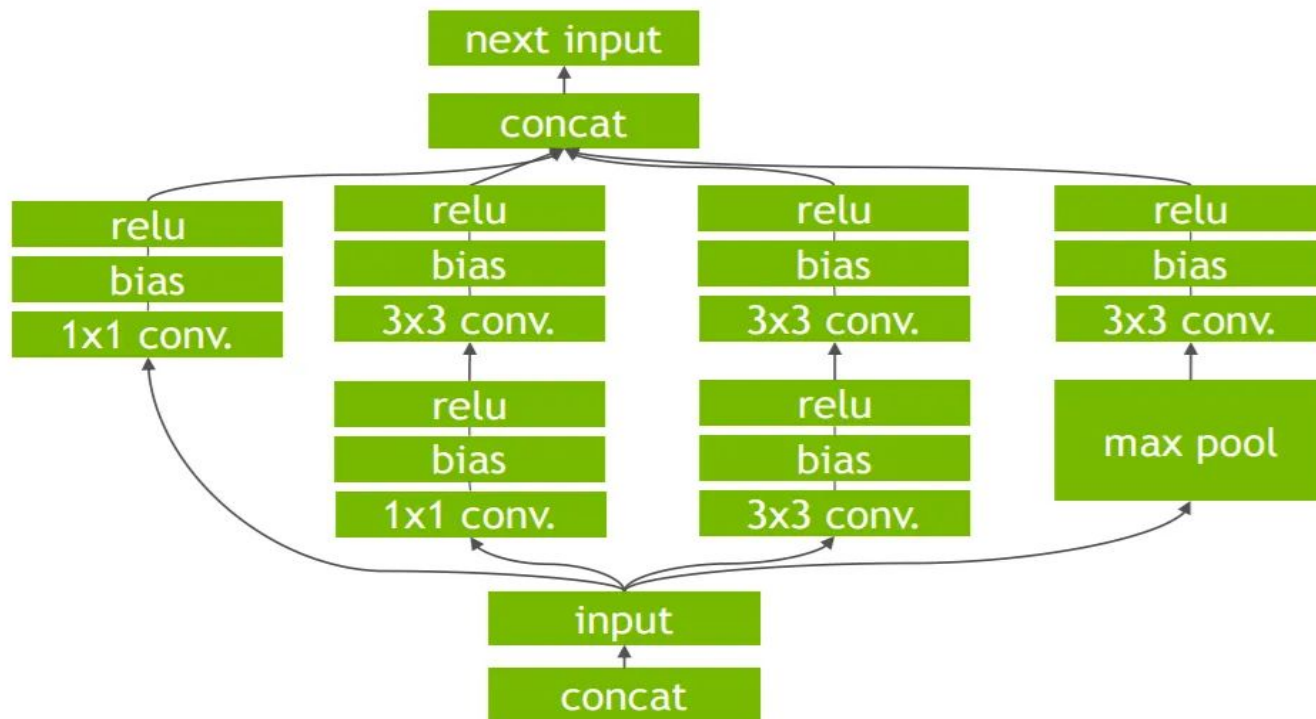


# TensorRT (aka TRT)



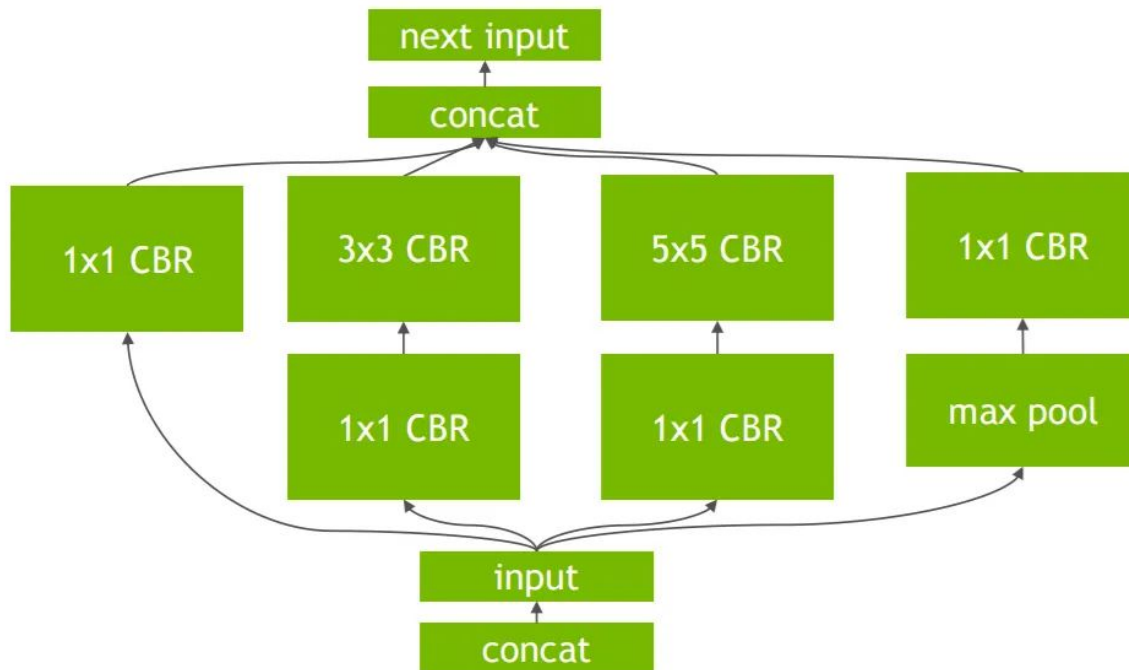
# TRT's Layer Fusion

DO

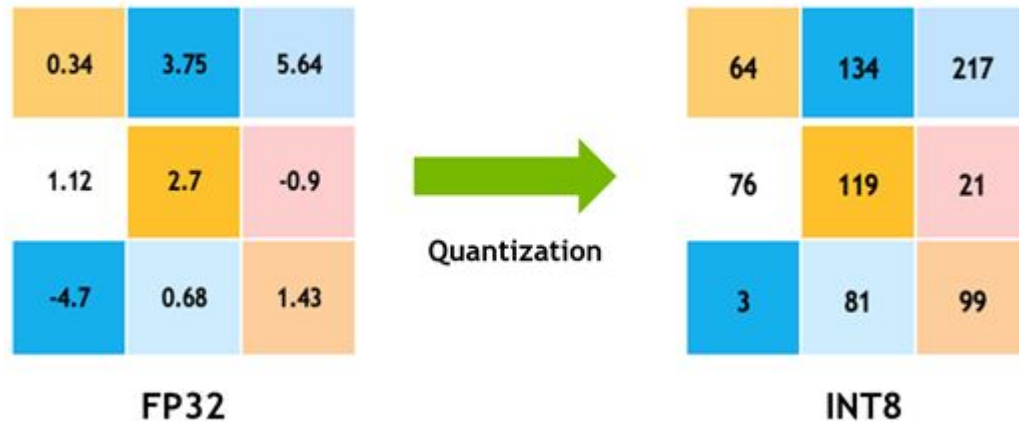


# TRT's Layer Fusion

## Vertical Fusion



# TRT's Static Quantization

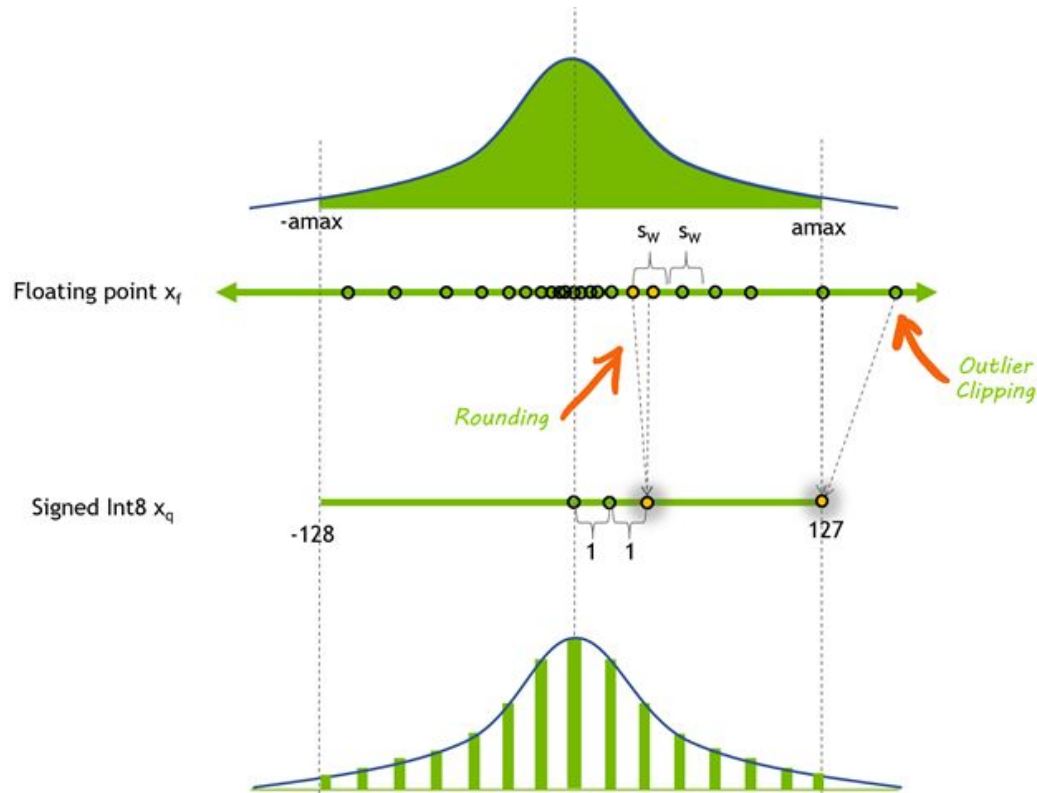


# TRT's Static Quantization

- INT8: [-127, 128]
- $x_f$ : FP32 Tensor
- $x_q$ : INT8 Tensor

Symmetric:  $x_q = \text{Clip}(\text{Round}(x_f / \text{scale}))$   
, где clip - throw outliers (-128, 127)

$$\text{scale} = 2 * \text{amax} / 256$$



# Не работает? QAT!



Figure 2. QAT fake-quantization operators in the training forward-pass (left) and backward-pass (right)

# Доп литература

- Accelerating Sparse Deep NN: <https://arxiv.org/pdf/2104.08378.pdf>
- Sparsity in INT8 for TRT:  
<https://developer.nvidia.com/blog/sparsity-in-int8-training-workflow-and-best-practices-for-tensorrt-acceleration/>
- Q/DQ Layer-Placement Recommendations:  
<https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/index.html#qdq-placement-recs>
- Dynamic shapes in TRT:  
[https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/index.html#work\\_dynamic\\_shapes](https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/index.html#work_dynamic_shapes)
- Custom Layers in TRT:  
<https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/index.html#extending>
- Deep Dive in TRT:  
<https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/index.html#work>

# Доп литература

- TensorRT-LLM: <https://github.com/NVIDIA/TensorRT-LLM>
- ONNX-Docs: <https://onnx.ai/onnx/>



# Спасибо за внимание!

Жду вопросов и обсуждений

